

Скидка до 50% и курс по ИИ в подарок

У нас есть классные рассылки:

Skillbox Media Код

КОД #СТАТЬИ

Компьютерное зрение OpenCV: где применяется и как работает в Python

Рассказываем, как компьютеры анализируют изображения, и учим их распознавать лица.

➦ Поделиться

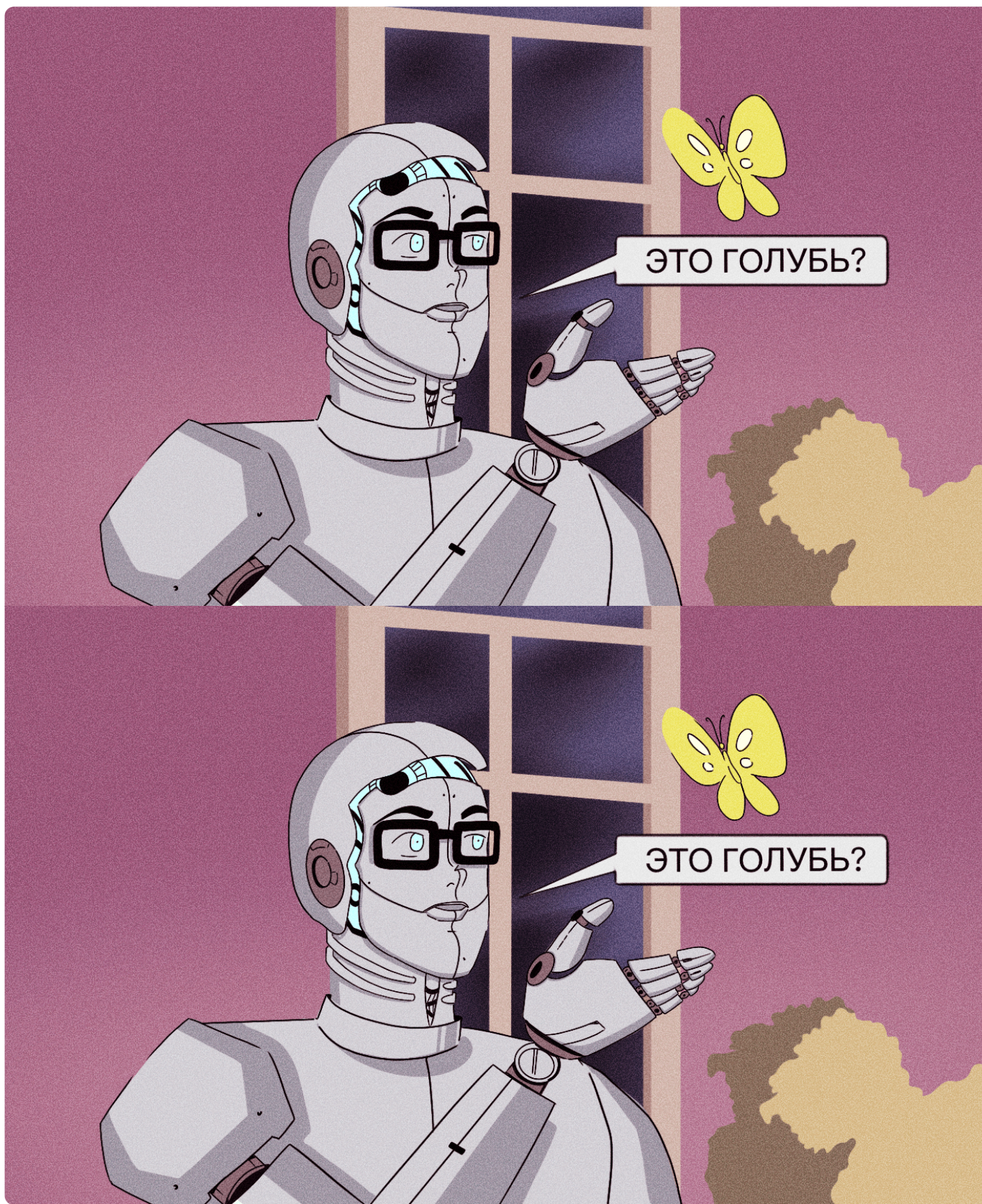


Иллюстрация: Оля Ежак для Skillbox Media



Иван Стуков

Журналист, изучает Python. Любит разбираться в мелочах, общаться с людьми и понимать их.



Курс с трудоустройством: «Профессия Data scientist + ИИ»

[Узнать о курсе](#)

Обучая маленьких детей, родители показывают им книжки с яркими цветными картинками: вот кошечка, вот цветочек, а вот машинка. И дети учатся отличать объекты друг от друга.

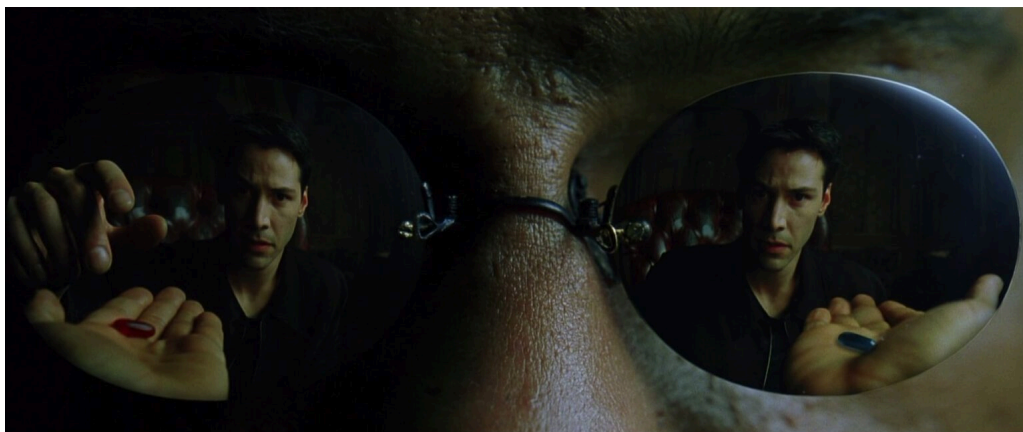
Компьютеры обучают распознавать изображения похожим образом — только изображений показывают в тысячи раз больше. Эта технология называется компьютерным зрением.

- [Как устроено компьютерное зрение](#)
- [Что такое OpenCV](#)
- [Где применяют компьютерное зрение и OpenCV](#)
- [Как работать с OpenCV в Python](#)
 - [Установка OpenCV](#)
 - [Как OpenCV видит изображения](#)
 - [Чтение, режимы отображения и запись](#)
 - [Изменение цвета пикселей](#)
 - [Изменение размера изображения](#)
 - [Обрезка](#)
 - [Поворот](#)
- [Примеры скриптов OpenCV](#)
 - [Отображение координат точки](#)
 - [Распознавание лиц](#)
- [Что дальше](#)

Как устроено компьютерное зрение

Компьютерное зрение — область искусственного интеллекта, связанная с получением, обработкой и анализом визуальной информации: изображений и видео.

Разберём определение на примере. Взгляните на эту картинку и опишите, что на ней изображено:



Кадр: фильм «Матрица» / Warner Bros.

Взгляд сразу выхватывает отдельные объекты. Тёмные очки, в одной линзе отражается человек, ладонь, красная таблетка, в другой — тот же человек, другая ладонь, синяя таблетка.

Если человек смотрел «Матрицу», он назовёт фильм, узнает персонажей и, возможно, даже вспомнит имена актёров — в зависимости от того, насколько он любит кинематограф и как много смотрел фильмов.

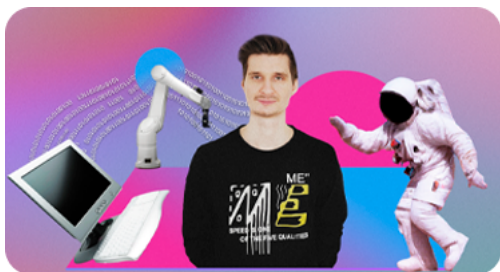
Что на этом изображении увидит программа, зависит от её бэкграунда, то есть от информации, которая у неё есть, и уровня сложности кода:

- Простенькая программа сможет только распознать, что это файл с расширением .jpg — то есть изображение.
- Программа посложнее откроет файл и поймёт, что он состоит из набора пикселей разного цвета. Она сможет вывести изображение на экран и, возможно, даже отредактировать: изменит цвет части пикселей на другой, обрежет картинку и так далее.
- Если программа работает с помощью нейронных сетей, то она проанализирует изображение и извлечёт из него дополнительную информацию, помимо технической: определит, где на картинке очки, руки, таблетки, лица, найдёт похожие изображения, узнает, из какого фильма кадр и какие актёры в нём играют. Это и есть компьютерное зрение. Набор извлечённой информации будет зависеть от того, чему программу учили.

Чтобы программа умела смотреть на изображение не как на набор пикселей, а как на набор известных ей объектов, используют **машинное обучение**.

Читайте также:

Что такое machine learning и стоит ли его изучать



Помимо компьютерного зрения, есть также **машинное зрение**. По сути, это то же компьютерное зрение, но применяется оно для решения конкретной прикладной задачи. Например, на производстве ставят камеру, которая следит за качеством продукции на конвейере. Если такая камера увидит брак, то она предупредит об этом человека-оператора — и это единственная её задача. В этом случае компьютерное зрение можно назвать машинным зрением.



Освойте data science за 12 месяцев

На курсе Skillbox вы пройдёте полный цикл работы с данными: от Excel до машинного обучения. Добавьте в портфолио кейсы от «Сбера».

[Узнать о курсе](#)

Что такое OpenCV

OpenCV (Open Source Computer Vision Library) — это библиотека с открытым исходным кодом для работы с компьютерным зрением. Изначально она была написана на C++, но адаптирована для использования на Python, C, Java и MATLAB.

В OpenCV есть встроенные алгоритмы компьютерного зрения на основе машинного обучения в виде отдельных модулей с разной функциональностью. Вот некоторые из них:

- **Core Functionality** (основная функциональность) — определяет основные структуры данных и функции библиотеки, которые используются в других модулях.

- **Image Processing** (обработка изображений) — позволяет работать со статичными изображениями: простыми картинками в форматах PNG, JPG и других.
- **Video Analysis** (анализ видео) — используется для отслеживания движений объектов и работы с фоном.
- **Camera Calibration and 3D Reconstruction** (калибровка камеры и 3D-реконструкция) — работает с геометрией объектов, позволяя создавать их 3D-модели на основе нескольких изображений или видео.
- **2D Features Framework** (фреймворк двумерных особенностей) — определяет фрагменты изображения, которые отличаются от других, запоминая их контуры, и может находить похожие среди них.
- **Object Detection** (обнаружение объектов) — находит объекты, например лица, автомобили, птиц и другое.
- **High-level GUI** (высокоуровневый графический интерфейс) — позволяет рисовать графические интерфейсы для выполнения простых операций.
- **Video I/O** (ввод и вывод видео) — позволяет считывать и обрабатывать видеофайлы.

Это только часть модулей OpenCV. Подробнее о них и о других возможностях библиотеки можно прочитать в [документации](#).

Где применяют компьютерное зрение и OpenCV

Компьютерное зрение уже повсюду в нашей жизни: роботы-пылесосы и автомобили объезжают препятствия, а соцсети находят фотографии с вами и спрашивают, вы ли это.

Компьютерное зрение используют для решения разных задач.

Распознавать текст. Например, чтобы перевести этикетку, можно включить на телефоне приложение-переводчик и навести его на текст. Приложение распознает его и переведёт на нужный язык.

Искать по изображениям. Можно сфотографировать незнакомый цветок и узнать его название, загрузив фотографию в поисковик Google или «Яндекс».

Модерировать контент. Многие форумы и соцсети анализируют контент и автоматически удаляют его, если он нарушает правила платформы.

Использовать биометрию. Чтобы вы могли разблокировать телефон или войти в сервис по отпечатку пальца или лицу, ваши данные обрабатываются алгоритмами компьютерного зрения.

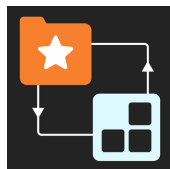
Создавать 3D-модели. Компьютерное зрение позволяет на основе нескольких фотографий или видео создать 3D-модель объекта или пространства. Это используется для восстановления облика объектов и ландшафта, а в робототехнике — чтобы робот мог ориентироваться в помещениях.

Развивать сельское хозяйство. С помощью компьютерного зрения можно считать поголовье скота и следить за посевами: оценивать их состояние, обнаруживать вредителей и болезни до того, как их заметил бы человек.

Управлять производством. Вместо того чтобы полагаться в выполнении рутинных задач на людей, компьютерное зрение может анализировать продукцию и процессы самостоятельно. Например, можно установить камеру, которая будет прямо на конвейере выявлять бракованную продукцию. В отличие от человека, такая система не устанет и не отвлечётся.

Для создания систем с компьютерным зрением подходят разные инструменты. Один из самых популярных — OpenCV. Его используют Google, Yahoo, Microsoft, Intel, IBM, Sony, Honda, Toyota и другие технологические компании для решения разнообразных задач.

Например, с помощью OpenCV в Китае следят за состоянием оборудования в шахтах, а в Японии распознают лица людей. Написать программу, распознающую лица, вы можете и сами. О том, как сделать это, [рассказываем ниже](#).



Курс «Профессия Data scientist + ИИ»

Изучите востребованную специальность на основе реальных проектов и добавьте в портфолио 20+ работ.

[Узнать о курсе](#)

Как работать с OpenCV в Python

Библиотека OpenCV бесплатна и доступна для использования в личных, образовательных и коммерческих целях. Рассмотрим её базовые функции на примере Python и в итоге напишем скрипт, распознающий лица людей.

Установка OpenCV

Чтобы запустить OpenCV, у вас должен быть установлен поддерживаемый ею язык программирования. В нашем случае это Python. Если у вас он ещё не установлен, можете воспользоваться [нашим руководством](#).

Скачать библиотеку можно с помощью инструментов вашей IDE или с помощью командной строки:

```
# Windows
pip install opencv-python

# macOS
brew install opencv3 --with-contrib --with-python3

# Linux
sudo apt-get install libopencv-dev python-opencv
```

Теперь можно импортировать модуль и приступить к работе:

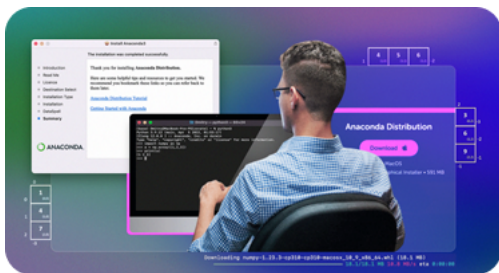
```
import cv2
```

Как OpenCV видит изображения

Перед тем как переходить к практике, остановимся на том, как OpenCV воспринимает изображения. Библиотека работает с ними как с [NumPy-массивами](#).

Читайте также:

Библиотека NumPy: всё, что нужно знать новичку



Если изображение в оттенках серого, то массив этот двумерный. Каждый пиксель в изображении представлен в виде числа 0 до 255, где 0 — чёрный, 255 — белый, а всё остальное — оттенки серого между ними.

Каждая пиксельная строка объединена в одномерный массив. Например, `[255, 255, 77, 77, 77, 255, 255]`. Это строка из семи пикселей: двух белых, трёх серых и ещё двух белых.

Каждая такая пиксельная строка объединена в массив второго уровня. Например:

```
[[255, 255, 255, 77, 255, 255, 255],
 [255, 255, 77, 77, 77, 255, 255],
 [255, 77, 77, 255, 77, 77, 255],
 [255, 255, 77, 77, 77, 255, 255]
 [255, 255, 255, 77, 255, 255, 255]]
```

В итоге получается вот такое изображение, только в 100 раз меньше:



Изображение: Skillbox Media

Если изображение цветное, то потребуется трёхмерный массив. Цвет каждого пикселя описывает не одно, а сразу три числа от 0 до 255 — в соответствии с [моделью RGB](#). Только красный и синий цвет переставлены местами, так что первое число отвечает за синий цвет, второе — за зелёный, третье — за красный: получается BGR.

Вот примеры пикселей разного цвета на языке OpenCV:

- [0, 0, 0] — чёрный.
- [255, 255, 255] — белый.
- [255, 0, 0] — синий.
- [0, 255, 0] — зелёный.
- [0, 0, 255] — красный.
- [0, 255, 255] — жёлтый.
- [203, 192, 255] — розовый.

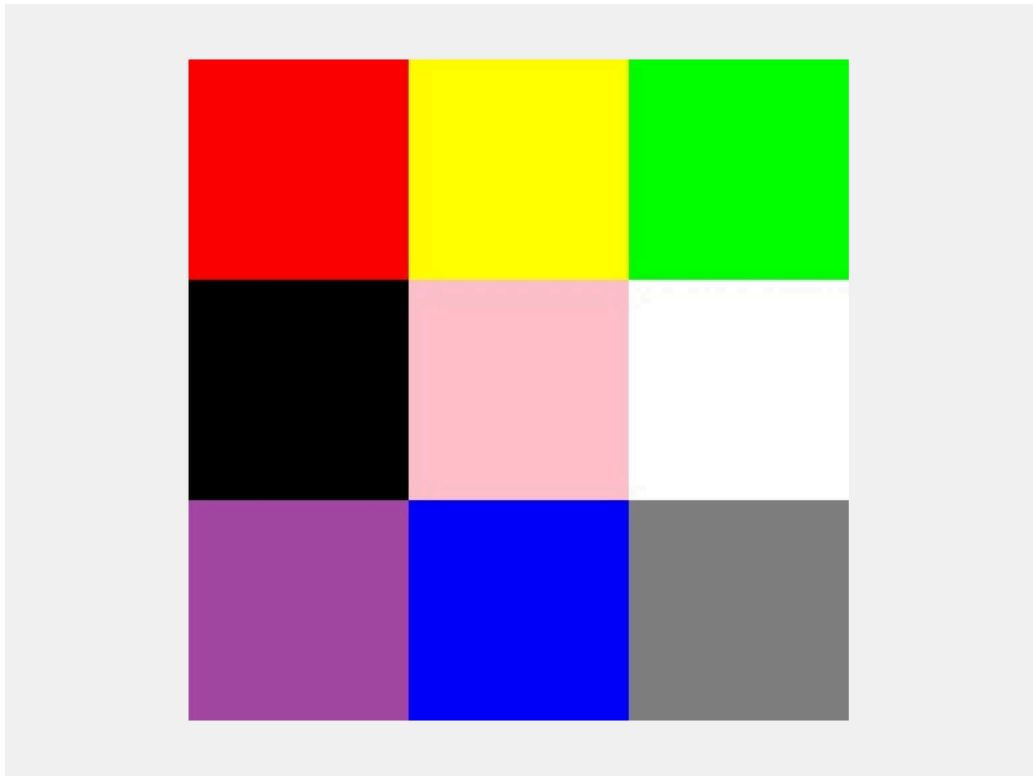
Таблицу распространённых цветов в RGB можно посмотреть [здесь](#). Только помните, что в OpenCV первый и третий цвета переставлены местами.

Как и в случае с изображениями в оттенках серого, массив следующего уровня объединяет строки пикселей, а верхнеуровневый массив — всё изображение.

Вот так в OpenCV будет выглядеть массив для изображения из девяти цветных пикселей:

```
[[[0, 0, 255], [0, 255, 255], [0, 255, 0]],  
 [[0, 0, 0], [203, 192, 255], [255, 255, 255]],  
 [[164, 73, 163], [255, 0, 0], [127, 127, 127]]]
```

И само изображение, увеличенное для удобства:

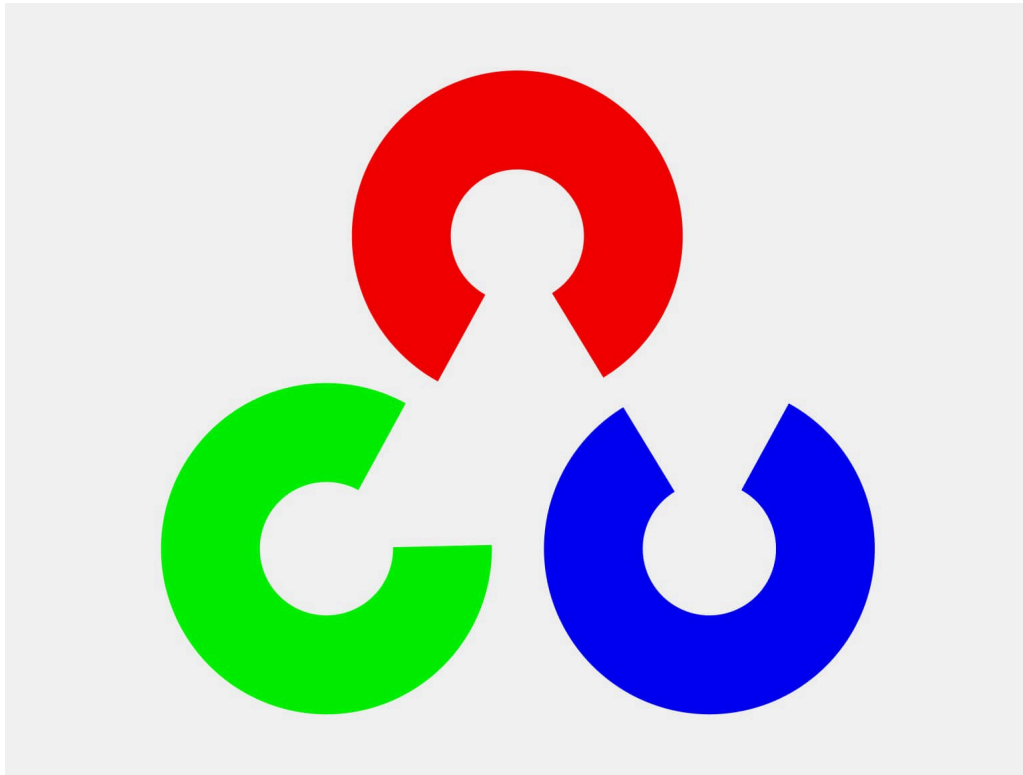


Изображение: Skillbox Media

Чтение, режимы отображения и запись

Скачаем любое изображение (желательно яркое и контрастное) и откроем его в нашем коде.

Например, вот такое изображение:



Изображение: OpenCV

Прочитайте изображение и откройте его в отдельном окне:

```
image = cv2.imread('logo.jpg')
cv2.imshow('logo', image)
cv2.waitKey(0)
```

- `cv2.imread` — «читает» изображение и возвращает NumPy-массив, с которым библиотека может работать. В качестве аргумента получает полный или относительный путь к изображению.
- `cv2.imshow` — открывает изображение в отдельном окне операционной системы. Первым аргументом получает название окна, вторым — NumPy-массив, в нашем случае он хранится в переменной `image`.
- `cv2.waitKey` — если не указать эту функцию, то открытое окно тут же закроется. В качестве аргумента получает кнопку, которую нужно нажать, чтобы закрыть окно. Если указать 0, то окно закроется при нажатии любой кнопки.

Можно передать функции `cv2.imread` необязательный второй аргумент и выбрать режим отображения:

- `cv2.IMREAD_COLOR` (по умолчанию) — цветное отображение в формате RGB.
- `cv2.IMREAD_GRAYSCALE` — отображение в оттенках серого.

Таким образом, функцию `cv2.imread` можно использовать, чтобы обесцветить цветное изображение:

```
gray_image = cv2.imread('logo.jpg', cv2.IMREAD_GRAYSCALE)
cv2.imshow('logo', gray_image)
cv2.waitKey(0)
```

Вот что получится:



Изображение: Skillbox Media

Чтобы сохранить новое изображение в отдельный файл, используйте функцию `cv2.imwrite` :

```
cv2.imwrite('gray_logo.jpg', gray_image)
```

Первым аргументом функция принимает название нового файла с картинкой, вторым — NumPy-массив с данными изображения.

Изменение цвета пикселей

OpenCV позволяет обращаться к пикселям по их записи в NumPy-массиве. Так мы можем менять изображение.

Для начала **получим доступ к пикселю**. Для этого обратимся к нему по координатам. Сначала указываем координаты ширины, потом — высоты. Например:

```
blue, green, red = image[10, 350]
print(f'RGB-код пикселя: {red}, {green}, {blue}')
>>> RGB-код пикселя: 249, 7, 6
```

Теперь **изменим цвет пикселя**. Для этого обратимся к нему по координатам и сделаем его синим. Помните, что синий и красный переставлены местами:

```
image[0, 0] = [255, 0, 0]
```

Если открыть изменённое изображение, то в правом верхнем углу вы увидите чёрную точку размером с один пиксель.

При ручной замене полезно знать размеры изображения. Для этого воспользуйтесь функцией `image.shape` :

- `image.shape[0]` — высота изображения в пикселях.

- `image.shape[1]` — ширина изображения в пикселях.

Используя циклы, можно изменять цвет сразу множества пикселей. Например, нарисовать для картинки чёрную рамку шириной 10 пикселей.

Внешние циклы перебирают каждый пиксель по горизонтали (первый) и вертикали (второй), а внутренние циклы идут от краёв к центру изображения и на каждом витке перекрашивают в чёрный по пикселю.

```
import cv2

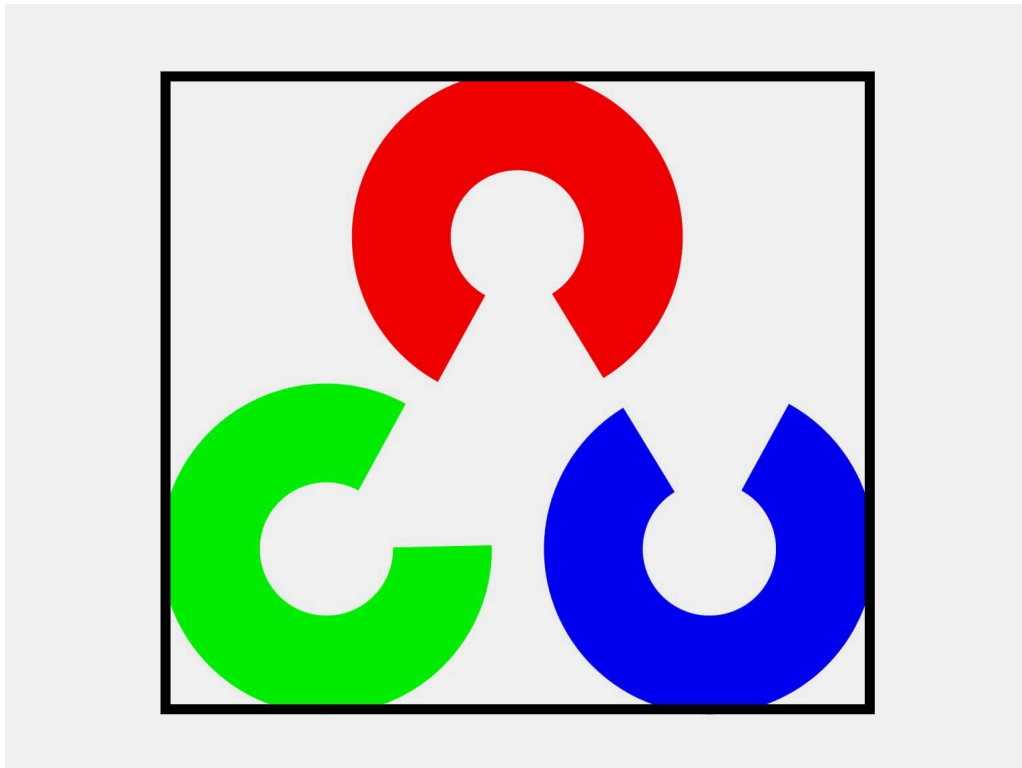
image = cv2.imread('logo.jpg')

# рисуем верхнюю и нижнюю линии
width = image.shape[1]
for width_coordinate in range(width):
    for frame_coordinate in range(0, 10):
        image[frame_coordinate, width_coordinate] = [0, 0, 0]
        image[-frame_coordinate, width_coordinate] = [0, 0, 0]

# рисуем правую и левую линии
height = image.shape[0]
for height_coordinate in range(height):
    for frame_coordinate in range(0, 10):
        image[height_coordinate, frame_coordinate] = [0, 0, 0]
        image[height_coordinate, -frame_coordinate] = [0, 0, 0]

# создаём новое окно с изменённым изображением
cv2.imshow('logo', image)
cv2.waitKey(0)
```

В итоге получается такое изображение:



Изображение: Skillbox Media

Изменение размера изображения

Чтобы изменить размер изображения, используйте функцию `cv2.resize`. Уменьшим нашу картинку в два раза:

```
reduced_image = cv2.resize(image, (399, 368), cv2.INTER_AREA)
```

- `image` — NumPy-массив изображения, размер которого мы изменяем.
- `(399, 368)` — ширина и высота нового изображения.
- `cv2.INTER_AREA` — метод интерполяции, то есть алгоритм, по которому OpenCV выбирает, в какие цвета красить пиксели.

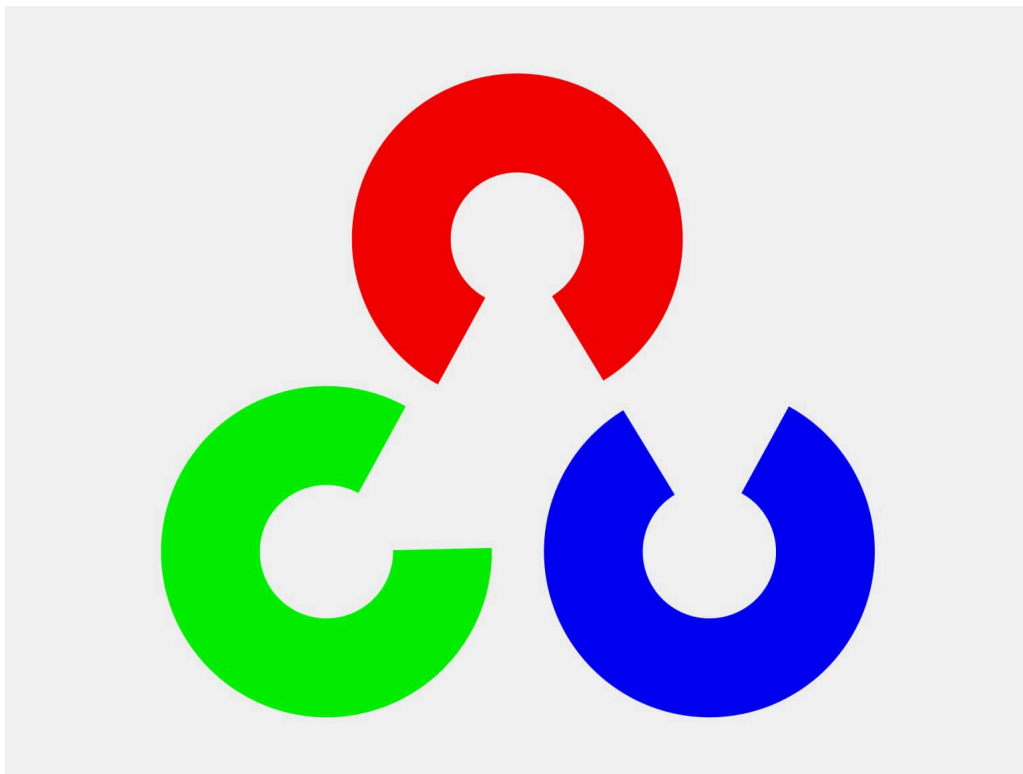
Есть несколько методов [интерполяции](#), но в большинстве случаев достаточно двух из них:

- `cv2.INTER_LINEAR` — для увеличения изображения. Используется по умолчанию, если не указано иное.
- `cv2.INTER_AREA` — для уменьшения картинки.

Чтобы сохранять пропорции изображения и каждый раз не высчитывать их вручную, используйте данные, полученные с помощью метода `shape`:

```
height = image.shape[0]
width = image.shape[1]
reduced_image = cv2.resize(image, (int(width/2), int(height/2)), cv2.INTER_AREA)
```

В обоих случаях результат одинаковый:



Изображение: Skillbox Media

Обрезка

Чтобы обрезать изображение, укажите диапазоны координат, в которых хотите оставить изображение, — сначала высоту, потом ширину. Например:

```
cropped_image = image[0:368, 200:600]
```



Изображение: Skillbox Media

Поворот

Чтобы повернуть картинку, используйте сразу две функции:

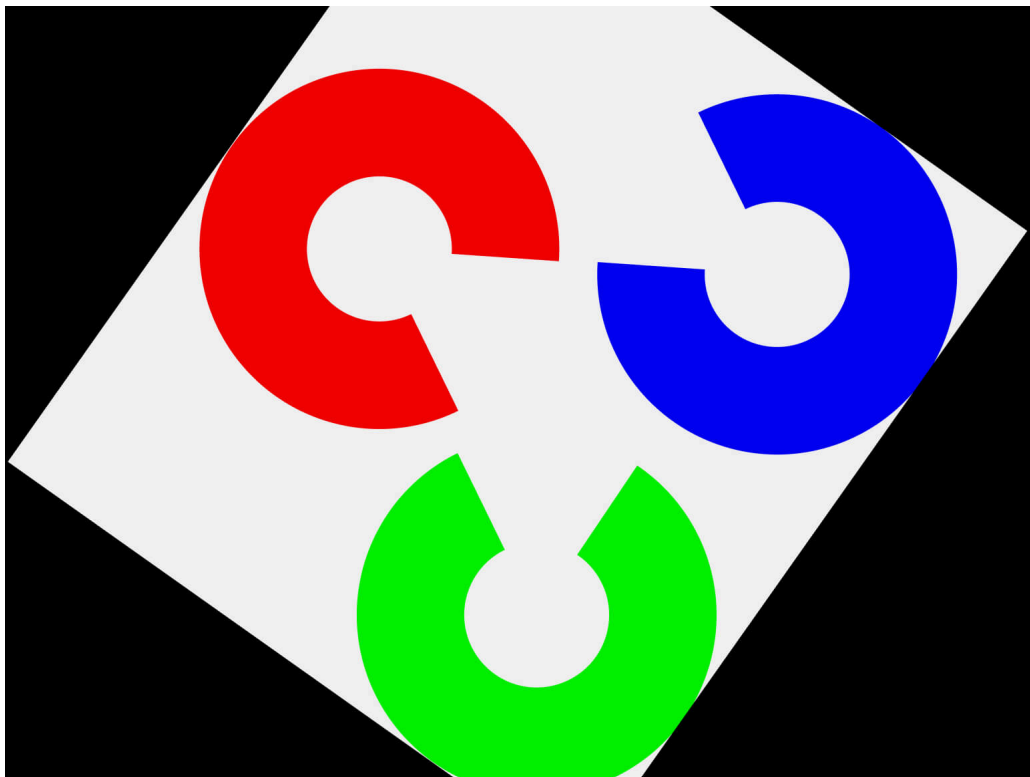
```
matrix = cv2.getRotationMatrix2D((399, 368), 60, 0.8)
rotated_image = cv2.warpAffine(image, matrix, (image.shape[1], image.shape[0]))
```

Функция `cv2.getRotationMatrix2D` задаёт матрицу, по которой будет повернуто изображение. Она получает следующие аргументы:

- `(399, 368)` — координаты точки, вокруг которой происходит поворот.
- `60` — угол поворота в градусах.
- `0.8` — коэффициент масштабирования. В нашем случае мы уменьшаем изображение, чтобы части логотипа не вышли за край.

Функция `cv2.warpAffine` непосредственно поворачивает изображение. Она получает следующие аргументы:

- `image` — само изображение.
- `matrix` — матрица, созданная функцией `cv2.getRotationMatrix2D`.
- `(image.shape[1], image.shape[0])` — размеры итогового изображения. В нашем случае мы оставляем те же, что были.



Изображение: Skillbox Media

Примеры скриптов OpenCV

Напишем пару простых скриптов для работы с изображениями, используя библиотеку OpenCV для Python.

Отображение координат точки

OpenCV умеет работать с нажатиями на открытое в окне изображение. Сделаем так, чтобы при нажатии на картинку программа выводила в терминал координаты точки и её цветовой код в формате BGR.

Дополнительно будем рисовать данные о точке, на которую нажали, прямо на изображении. Если нажали левую кнопку мыши — координаты, если правую — BGR-код.

Напишите функцию `click_event`, которая будет реагировать на нажатие на картинку:

```
def click_event(event, x, y, flags, params):

    # если нажата левая кнопка мыши
    if event == cv2.EVENT_LBUTTONDOWN:
        pass # заглушка вместо будущего кода

    # если нажата правая кнопка мыши
    if event == cv2.EVENT_RBUTTONDOWN:
        pass # заглушка вместо будущего кода
```

Функция `click_event` принимает пять аргументов:

- `event` — произошедшее событие (в нашем случае нажатие кнопки мыши).
- `x` и `y` — координаты точки, на которую нажали.
- `flags` и `params` — служебные аргументы, нужные методу, с помощью которого мы будем вызывать эту функцию.

При нажатии любой кнопки мыши мы хотим выводить в терминал координаты и BGR-код точки. Пропишите это:

```
def click_event(event, x, y, flags, params):

    # если нажата левая кнопка мыши
    if event == cv2.EVENT_LBUTTONDOWN:
        # узнаём BGR-код точки и сохраняем в переменные
        b = img[y, x, 0]
        g = img[y, x, 1]
        r = img[y, x, 2]

        # выводим координаты и BGR-код точки в терминал
        print(f'Координаты точки: {x}, {y}')
        print(f'BGR-код точки: {b}, {g}, {r}\n')

    # если нажата правая кнопка мыши
    if event == cv2.EVENT_RBUTTONDOWN:
        # узнаём BGR-код точки и сохраняем в переменные
        b = img[y, x, 0]
        g = img[y, x, 1]
        r = img[y, x, 2]

        # выводим координаты и BGR-код точки в терминал
```

```
print(f'Координаты точки: {x}, {y}')
print(f'BGR-код точки: {b}, {g}, {r}\n')
```

Дополните первое условие кодом, который будет рисовать координаты точки прямо на изображении, если нажата левая кнопка мыши:

```
# устанавливаем шрифт, которым будем писать надпись
font = cv2.FONT_HERSHEY_SIMPLEX
# пишем надпись
cv2.putText(img, f'{x}, {y}', (x, y),
            font, 1, (0, 0, 0), 2)
# отображаем изменённое изображение
cv2.imshow('image', img)
```

Функция `cv2.putText` принимает следующие аргументы:

- `img` — изображение, на которое добавляется надпись.
- `f'{x}, {y}'` — надпись, которая добавляется на изображение.
- `(x, y)` — координаты точки, из которой выходит надпись.
- `font` — шрифт надписи.
- `1` — масштабирование размера текста.
- `(0, 0, 0)` — цвет текста.
- `2` — толщина линий.

По аналогии дополните второе условие кодом, который будет рисовать на изображении BGR-код, если нажата правая кнопка мыши:

```
# устанавливаем шрифт, которым будем писать надпись
font = cv2.FONT_HERSHEY_SIMPLEX
# пишем надпись
cv2.putText(img, f'{b}, {g}, {r}', (x, y),
            font, 1, (0, 0, 0), 2)
# отображаем изменённое изображение
cv2.imshow('image', img)
```

Наша функция готова. Остаётся только передать изображение и воспользоваться встроенной OpenCV-функцией `cv2.setMouseCallback`:

```
if __name__ == "__main__":
    img = cv2.imread('logo.jpg', 1)
    cv2.imshow('image', img)
    # устанавливаем реакцию на действия мыши и вызываем функцию click_event
    cv2.setMouseCallback('image', click_event)
    cv2.waitKey(0)
```

Итоговый код:

```
import cv2

# функция реакции на нажатие левой или правой кнопки мыши
def click_event(event, x, y, flags, params):

    # если нажата левая кнопка мыши
    if event == cv2.EVENT_LBUTTONDOWN:
        # выводим координаты и BGR-код точки в терминал
```

```

print(f'Координаты точки: {x}, {y}')

b = img[y, x, 0]
g = img[y, x, 1]
r = img[y, x, 2]
print(f'BGR-код точки: {b}, {g}, {r}\n')

# рисуем координаты точки на изображении
font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img, f'{x}, {y}', (x, y),
            font, 1, (0, 0, 0), 2)
cv2.imshow('image', img)

# если нажата правая кнопка мыши
if event == cv2.EVENT_RBUTTONDOWN:
    # узнаём BGR-код точки и сохраняем в переменные
    b = img[y, x, 0]
    g = img[y, x, 1]
    r = img[y, x, 2]

    # выводим координаты и BGR-код точки в терминал
    print(f'Координаты точки: {x}, {y}')
    print(f'BGR-код точки: {b}, {g}, {r}\n')

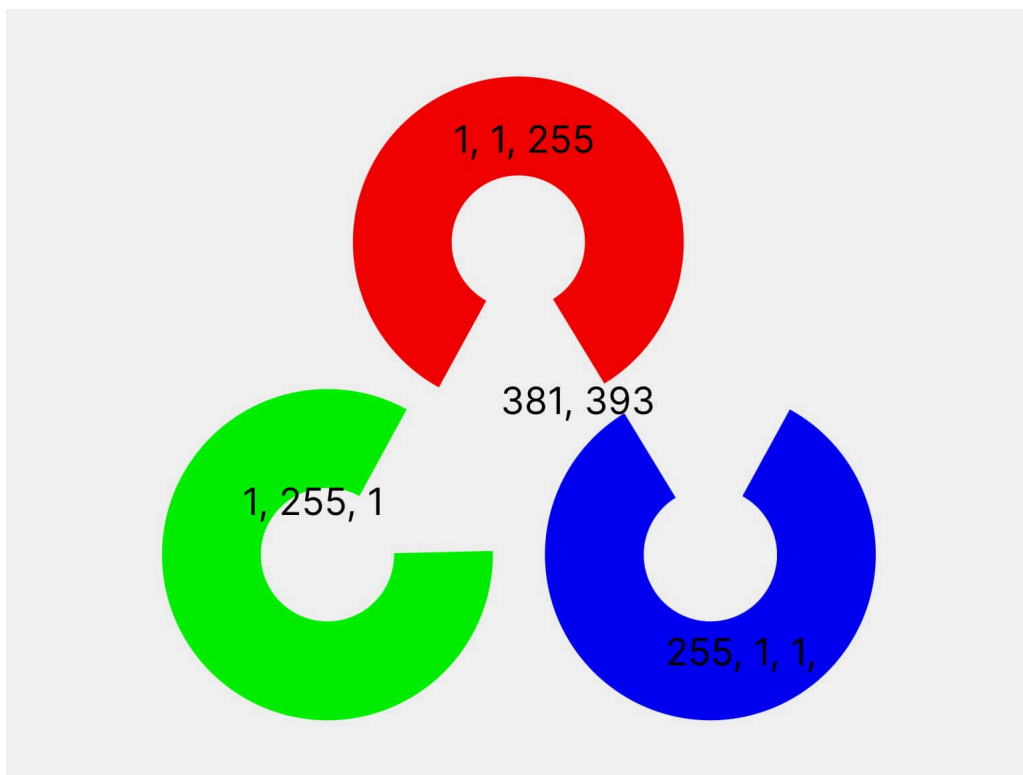
    # рисуем BGR-код точки на изображении
    font = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(img, f'{b}, {g}, {r}', (x, y),
                font, 1, (0, 0, 0), 2)
    cv2.imshow('image', img)

# основной сценарий
if __name__ == "__main__":
    img = cv2.imread('logo.jpg', 1)
    cv2.imshow('image', img)

    # устанавливаем реакцию на действия мыши и вызываем функцию click_event
    cv2.setMouseCallback('image', click_event)
    cv2.waitKey(0)

```

Опробуйте код. Откройте изображение, нажмите правой кнопкой мыши на каждую из фигур и левой — на пространство между ними. Получится вот такой результат:



Изображение: Skillbox Media

При этом в терминале выводится следующий текст:

```
>>> Координаты точки: 383, 68
>>> BGR-код точки: 1, 1, 255

>>> Координаты точки: 68, 494
>>> BGR-код точки: 1, 255, 1

>>> Координаты точки: 579, 665
>>> BGR-код точки: 255, 1, 1

>>> Координаты точки: 381, 393
>>> BGR-код точки: 255, 255, 255
```

Распознавание лиц

В OpenCV есть и более «умные» инструменты, использующие машинное обучение. Один из них — [модель Haar](#), которая умеет распознавать лица.

Чтобы воспользоваться ей, скачайте [файл haarcascade_frontalface_default.xml](#), выложенный OpenCV на GitHub.

Также вам понадобится любое изображение с лицами людей. Мы воспользуемся кадром из «Матрицы», который уже показывали в начале этой статьи.



Кадр: фильм «Матрица» / Warner Bros.

Сохраните эти файлы в папку с вашим скриптом, и тогда к ним можно будет обращаться по имени.

Импортируйте OpenCV и создайте по переменной для модели Haar и изображения:

```
import cv2

# создаём переменную с файлом модели
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

# читаем изображение
image = cv2.imread('matrix.jpg')
```

При работе с распознаванием и сравнением объектов используют обесцвеченные версии изображений. Обесцветим и наше:

```
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Чтобы обнаружить на картинке лица, используйте метод `detectMultiScale`, который применим к модели Haar.

```
faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1, minNeighbors=5)
```

Мы передаём методу следующие аргументы:

- `gray_image` — обесцвеченное изображение;
- `scaleFactor=1.1` — шаг масштабирования изображения. Дело в том, что в модели хранятся данные о лицах определённого размера. Если лица на изображении больше или меньше, то алгоритм не обнаружит их. Поэтому при анализе размер изображения меняется: чтобы лица на нём в какой-то момент стали того же размера, что и в модели. Чем меньше `scaleFactor`, тем точнее распознавание, но тем медленнее оно работает;
- `minNeighbors=5` — какое минимальное число совпадений с моделью должно быть на изображении, чтобы признать объект лицом. Чем больше этот аргумент, тем меньше лиц алгоритм будет обнаруживать, но вместе с тем уменьшается шанс принять за лицо какой-либо другой объект.

Метод `detectMultiScale` возвращает координаты полученных объектов. Используйте эти координаты и функцию `cv2.rectangle`, чтобы нарисовать вокруг лиц квадраты:

```
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 0, 255), 2)
```

В переменной `faces` хранится список из четырёх элементов. Каждый из них — список с данными всех найденных объектов:

- x — координаты верхнего левого угла объекта по горизонтали.
- y — координаты верхнего левого угла объекта по вертикали.
- w — ширина объекта.
- h — высота объекта.

Функция `cv2.rectangle` получает следующие аргументы:

- `image` — цветное изображение, на котором мы рисуем квадрат.
- (x, y) — координаты верхнего левого угла квадрата.
- $(x+w, y+h)$ — координаты нижнего правого угла квадрата.
- $(0, 0, 255)$ — цвет квадрата в формате BGR.
- `2` — толщина линии квадрата в пикселях.

Посмотрим на получившееся изображение:

```
cv2.imshow('found_faces', image)
cv2.waitKey(0)
```

Итоговый код:

```
import cv2

# создаём переменную с файлом модели
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

# читаем изображение
image = cv2.imread('matrix.jpg')

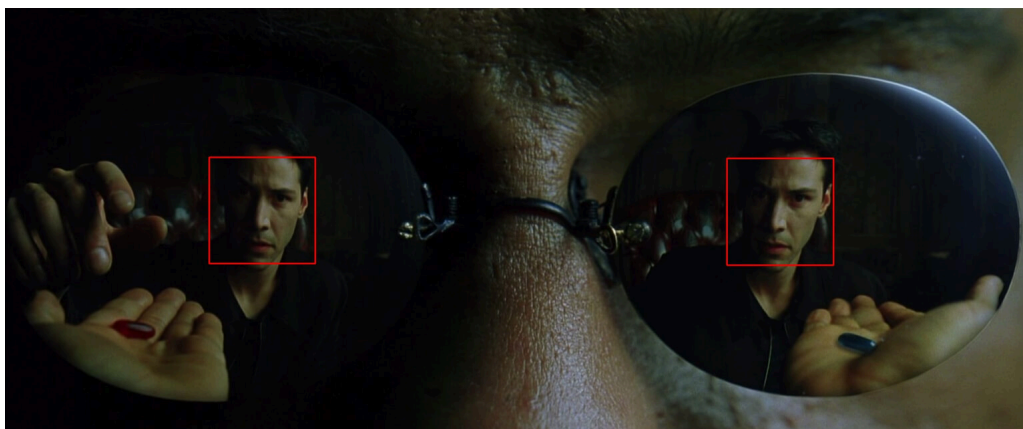
# обесцвечиваем изображение
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# находим лица на обесцвеченном изображении
faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1, minNeighbors=5)

# на цветном изображении рисуем квадраты там, где нашли лица на обесцвеченном
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 0, 255), 2)

# открываем изображение в отдельном окне
cv2.imshow('found_faces', image)
cv2.waitKey(0)
```

Результат:



Что дальше

Путешествие

Кадр: фильм «Матрица» / Warner Bros.

Чтобы глубже погрузиться в тему компьютерного зрения и OpenCV, вы также можете изучить: Всё получилось — лица в отражении обведены рамками.

- [туториалы из документации по OpenCV](#) (английский);
- [OpenCV Python Tutorial](#) от платформы GeeksforGeeks (английский);
- Анирад Коул, Мехер Казам, Сиддха Ганджу. «[Искусственный интеллект и компьютерное зрение](#)»;
- Эдуард Шакирьянов. «[Компьютерное зрение на Python](#)»;
- [OpenCV Course — Full Tutorial with Python](#) (видео, есть английские субтитры).

ЧИТАЙТЕ ТАКЖЕ:

- [Искусственный интеллект, машинное обучение и глубокое обучение: в чём разница](#)
- «[Эта профессия даёт свободу](#)»: история Python-разработчика Юрия Кожанова
- [Создаём «Змейку» — первую игру на Python и Pygame](#)

Больше интересного про код — в нашем [телеграм-канале](#).
Подписывайтесь!

➦ Поделиться

Курс с помощью в трудоустройстве

Профессия Data scientist + ИИ

- Спикеры — дата-сайентисты из топ-компаний: VK, ВТБ, «Сбера», Wildberries
- Обучение на реальных задачах от крупных проектов
- Нейросети в программе, чтобы быстрее собирать и обрабатывать данные

[Подробнее](#)



Новости

TIOBE обновил рейтинг языков программирования за апрель 2026-го

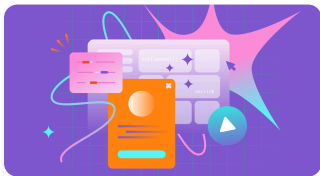
13 апр 2026

Anthropic ограничила доступ к новой ИИ-модели: нейросеть умеет взламывать IT-системы

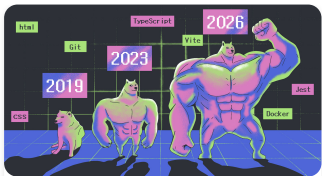
09 апр 2026

Очередной масштабный сбой в работе Рунета: что известно

06 апр 2026



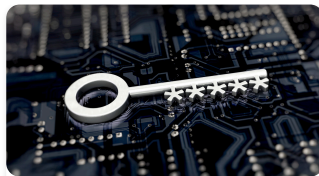
Что такое Set в JavaScript
и зачем он нужен, если есть
массивы



Стоит ли идти во фронтенд
в 2026 году, когда вокруг
нейросети и дикая
конкуренция?



Чем занимается
администратор баз данных
и как им стать



Как разблокировать телефон,
если забыли пароль

8 (800) 500-05-22

Контактный центр

+7(495) 291-59-87

Отдел заботы о пользователях

Москва, Ленинский проспект,
дом 6, строение 20

16+

[Договор оферты](#) [Оплата](#)

[Правила пользования Платформой](#)

[Политика конфиденциальности](#)